

```
In [2]: import pandas as pd

from sklearn.preprocessing import OrdinalEncoder
from sklearn.model_selection import train_test_split

from sklearn.naive_bayes import CategoricalNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
```

```
In [3]: df = pd.read_csv("Lab 8 - Sheet1.csv")

print("Dataset")
df.head()
```

Dataset

```
Out[3]:
```

	No	Outlook	Temperature	Humidity	Wind	Play Tennis
0	1	Sunny	Hot	High	Weak	No
1	2	Sunny	Hot	High	Strong	No
2	3	Overcast	Hot	High	Weak	Yes
3	4	Rain	Mild	High	Weak	Yes
4	5	Rain	Cool	Normal	Weak	Yes

```
In [4]: X = df[['Outlook', 'Temperature', 'Humidity', 'Wind']]

y = df['Play Tennis']

print("\nFeatures")
print(X)

print("\nTarget")
print(y)
```

Features

	Outlook	Temperature	Humidity	Wind
0	Sunny	Hot	High	Weak
1	Sunny	Hot	High	Strong
2	Overcast	Hot	High	Weak
3	Rain	Mild	High	Weak
4	Rain	Cool	Normal	Weak
5	Rain	Cool	Normal	Strong
6	Overcast	Cool	Normal	Strong
7	Sunny	Mild	High	Weak
8	Sunny	Cool	Normal	Weak
9	Rain	Mild	Normal	Weak
10	Sunny	Mild	Normal	Strong
11	Overcast	Mild	High	Strong
12	Overcast	Hot	Normal	Weak
13	Rain	Mild	High	Strong
14	Sunny	Cool	High	Weak
15	Sunny	Cool	High	Strong
16	Sunny	Hot	Normal	Weak
17	Sunny	Hot	Normal	Strong
18	Overcast	Cool	High	Weak
19	Overcast	Cool	High	Strong
20	Overcast	Mild	Normal	Weak
21	Overcast	Mild	Normal	Strong
22	Rain	Hot	High	Weak
23	Rain	Hot	High	Strong
24	Rain	Hot	Normal	Weak
25	Rain	Hot	Normal	Strong
26	Rain	Cool	High	Weak
27	Rain	Cool	High	Strong
28	Rain	Mild	Normal	Strong
29	Sunny	Mild	High	Strong
30	Sunny	Cool	Normal	Strong
31	Sunny	Hot	High	Weak
32	Sunny	Hot	Normal	Weak
33	Sunny	Mild	Normal	Weak
34	Overcast	Hot	High	Strong
35	Overcast	Hot	Normal	Strong
36	Overcast	Cool	Normal	Weak
37	Overcast	Mild	High	Weak
38	Rain	Mild	High	Weak
39	Rain	Cool	Normal	Weak
40	Rain	Hot	High	Strong
41	Sunny	Cool	High	Weak
42	Sunny	Mild	Normal	Strong
43	Sunny	Hot	High	Strong
44	Overcast	Cool	High	Weak
45	Overcast	Mild	Normal	Strong
46	Rain	Mild	Normal	Weak
47	Rain	Cool	High	Strong
48	Sunny	Cool	Normal	Weak
49	Overcast	Hot	Normal	Weak

Target

0	No
1	No
2	Yes
3	Yes
4	Yes
5	No
6	Yes
7	No
8	Yes
9	Yes

```
10  Yes
11  Yes
12  Yes
13  No
14  No
15  No
16  Yes
17  Yes
18  Yes
19  Yes
20  Yes
21  Yes
22  Yes
23  No
24  Yes
25  No
26  Yes
27  No
28  Yes
29  No
30  Yes
31  No
32  Yes
33  Yes
34  Yes
35  Yes
36  Yes
37  Yes
38  Yes
39  Yes
40  No
41  No
42  Yes
43  No
44  Yes
45  Yes
46  Yes
47  No
48  Yes
49  Yes
```

Name: Play Tennis, dtype: object

The model will use the weather conditions to predict whether a person will play tennis.

```
In [5]: encoder = OrdinalEncoder()

X_encoded = encoder.fit_transform(X)

print("\nEncoded Features")
print(X_encoded[:3])
```

```
Encoded Features
[[2. 1. 0. 1.]
 [2. 1. 0. 0.]
 [0. 1. 0. 1.]]
```

```
In [6]: X_train, X_test, y_train, y_test = train_test_split(
        X_encoded,
        y,
        test_size=0.3,
        random_state=42,
        stratify=y
    )
```

```
print("\nTraining Samples:", len(X_train))
print("Testing Samples:", len(X_test))
```

Training Samples: 35

Testing Samples: 15

The dataset is divided into training and testing sets using a 70:30 ratio, and stratified so as to ensure equal representation of each class.

Categorical Naive Bayes Classifier

```
In [7]: nb_model = CategoricalNB()

nb_model.fit(X_train, y_train)
```

Out[7]: CategoricalNB()

```
In [8]: y_pred = nb_model.predict(X_test)

print("\nPredicted Labels")
print(y_pred)
```

Predicted Labels

```
['Yes' 'No' 'Yes' 'No' 'Yes' 'Yes' 'Yes' 'Yes' 'Yes' 'Yes' 'Yes' 'Yes'
 'Yes' 'Yes' 'No']
```

```
In [9]: accuracy = accuracy_score(y_test, y_pred)

print("\nNaive Bayes Accuracy =", accuracy)
```

Naive Bayes Accuracy = 0.8666666666666667

The Categorical Naive Bayes model achieved an accuracy of 86.67%, meaning it correctly classified approximately 87 out of every 100 test samples.

```
In [10]: cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix")
print(cm)
```

Confusion Matrix

```
[[ 3  2]
 [ 0 10]]
```

There were only 2 misclassifications out of 15

```
In [11]: report = classification_report(y_test, y_pred)

print("\nClassification Report")
print(report)
```

Classification Report

	precision	recall	f1-score	support
No	1.00	0.60	0.75	5
Yes	0.83	1.00	0.91	10
accuracy			0.87	15
macro avg	0.92	0.80	0.83	15
weighted avg	0.89	0.87	0.86	15

```
In [12]: new_data = pd.DataFrame({
        'Outlook': ['Sunny'],
        'Temperature': ['Cool'],
        'Humidity': ['High'],
        'Wind': ['Strong']
    })

    new_encoded = encoder.transform(new_data)

    prediction = nb_model.predict(new_encoded)

    print("\nPrediction:")
    print(prediction[0])
```

Prediction:
No

For the given weather condition (Sunny, Cool, High Humidity, Strong Wind), the model predicts that the person will not play tennis, as the conditions were deemed unfavourable.

```
In [13]: probability = nb_model.predict_proba(new_encoded)

    print("\nPrediction Probability")

    print("No =", probability[0][0])
    print("Yes =", probability[0][1])
```

Prediction Probability
No = 0.746606334841629
Yes = 0.2533936651583709

The model assigns a probability of 74.66% to the "No" class and 25.34% to the "Yes" class. Since the probability of "No" is higher, the final prediction is No.

Decision Tree Model

```
In [14]: dt_model = DecisionTreeClassifier(random_state=42)

    dt_model.fit(X_train, y_train)

    dt_pred = dt_model.predict(X_test)

    dt_accuracy = accuracy_score(y_test, dt_pred)

    print("\nDecision Tree Accuracy =", dt_accuracy)
```

Decision Tree Accuracy = 1.0

The Decision Tree classifier achieved 100% accuracy, correctly classifying all test samples.

```
In [20]: dt_prediction = dt_model.predict(new_encoded)

    print("\nDecision Tree Prediction:")
    print(dt_prediction[0])
    dt_probability = dt_model.predict_proba(new_encoded)

    print("\nDecision Tree Probability")

    for cls, prob in zip(dt_model.classes_, dt_probability[0]):
        print(cls, ":", prob)
```

Decision Tree Prediction:
No

Decision Tree Probability
No : 1.0
Yes : 0.0

Logistic Regression Model

```
In [16]: lr_model = LogisticRegression(max_iter=200)

lr_model.fit(X_train, y_train)

lr_pred = lr_model.predict(X_test)

lr_accuracy = accuracy_score(y_test, lr_pred)

print("\nLogistic Regression Accuracy =", lr_accuracy)
```

Logistic Regression Accuracy = 0.9333333333333333

The Logistic Regression classifier achieved 93.33% accuracy, indicating excellent predictive performance with only a small number of incorrect classifications.

```
In [19]: lr_prediction = lr_model.predict(new_encoded)

print("\nLogistic Regression Prediction:")
print(lr_prediction[0])
lr_probability = lr_model.predict_proba(new_encoded)

print("\nLogistic Regression Probability")

for cls, prob in zip(lr_model.classes_, lr_probability[0]):
    print(cls, ":", prob)
```

Logistic Regression Prediction:
No

Logistic Regression Probability
No : 0.8257277683809853
Yes : 0.17427223161901476

```
In [17]: print("\nModel Comparison")

print("Categorical Naive Bayes :", accuracy)

print("Decision Tree          :", dt_accuracy)

print("Logistic Regression     :", lr_accuracy)
```

Model Comparison
Categorical Naive Bayes : 0.8666666666666667
Decision Tree : 1.0
Logistic Regression : 0.9333333333333333

Since the dataset is relatively small, the perfect accuracy may also indicate that the Decision Tree has learned the specific patterns in this dataset very closely.

Logistic Regression achieved an accuracy of 93.33%, which indicates that it was able to effectively learn the relationship between weather conditions and the decision to play tennis.

The slightly lower accuracy of Naive Bayes may be because it assumes that the input features are independent of each other, while in real-world datasets some features such as temperature, humidity, and weather conditions may have relationships.